

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**РАЗРАБОТВАНЕ НА БИБЛИОТЕКА ЗА
ОБЕКТНО-РЕЛАЦИОННО КАРТОГРАФИРАНЕ
ПО ВРЕМЕ НА КОМПИЛАЦИЯ**

ДИПЛОМНА РАБОТА

Изготвил:

Алекс Цветанов, фак. № 121222225

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

проф. д-р инж. Румен Трифонов

София, 2026

СЪДЪРЖАНИЕ

I.	Увод	3
1.1.	Постановка на проблема	3
1.2.	Цел и задачи на разработката	4
1.3.	Обхват и ограничения	5
1.4.	Структура на изложението	5
II.	Обзор на съществуващи решения	7
2.1.	Класификация на подходите	7
2.2.	Критерии за сравнение	8
2.3.	Анализ на представителните решения	8
2.3.1.	Клиентски библиотеки на драйверите	8
2.3.2.	Съставители на заявки и типизирани SQL DSL	9
2.3.3.	Класически рамки за обектно-реляционно картографиране	10
2.4.	Обобщителна сравнителна таблица	10
2.5.	Позициониране на настоящата разработка	12
III.	Проектиране на библиотеката	13
3.1.	Теоретична основа на обектно-реляционното картографиране	13
3.2.	Модели за съхранение	14
3.3.	Архитектурни цели и слоева организация	16
3.4.	Обектен слой	17
3.5.	Слой на заявките	18
3.6.	Слой на конекторите и формален договор	19
3.7.	Механизъм на способностите	21
3.8.	Прототипен слой за миграции	23
3.9.	Теория на корутините и асинхронен дизайн	24
3.9.1.	Корутини спрямо нишки и процеси	24
3.9.2.	Awaitable договор и преобразуване в крайно автоматно представяне	25
3.9.3.	Task<T> като носител на жизнения цикъл	25
3.9.4.	Универсален и асинхронен път през интерфейса на драйвера	26
3.9.5.	Координация на връзките и транзакциите при асинхронен достъп	28

3.10. Архитектурни инварианти и обобщение	28
IV. Програмна реализация	30
4.1. Методика и среда за реализация	30
4.2. Реализация на асинхронния слой	30
4.2.1. Task<T>, продължение и завършване	31
4.2.2. ThreadPool и универсален път на изнасяне	31
4.2.3. IoContext и интеграция с неблокиращите интерфейси на драйверите	32
4.2.4. Обединение от връзки и транзакции при асинхронен достъп	32
4.3. Реализация на конекторите по системи за съхранение	33
4.3.1. SQLite	33
4.3.2. PostgreSQL	34
4.3.3. MySQL	35
4.3.4. MongoDB	35
4.3.5. Redis	36
4.3.6. Графов и ширококолонен сценарий (Neo4j и Cassandra)	37
4.3.7. Обобщителна матрица на способностите	37
4.4. Верификация и тестване	38
4.4.1. Стратегия на верификацията	39
4.4.2. Каталог на единичните и интеграционните тестове	40
4.4.3. Интеграционни и реално свързани с живи системи за съхранение тестове	41
4.4.4. Емпирични резултати	42
4.4.5. Покритие и ограничения на верификацията	44
V. Заключение	46
5.1. Обобщение на решената задача	46
5.2. Постигнати научни и приложни приноси	47
5.3. Ограничения на разработката	49
5.4. Бъдещо развитие	49
5.5. Значение и заключителни бележки	51

РЕЗЮМЕ

Настоящата дипломна работа разглежда проектирането, реализацията и оценяването на библиотека за обектно-релационно картографиране по време на компилация (compile-time Object-Relational Mapping, ORM) за C++23 [1]. Работата е организирана около две допълващи се оси. Първата е статичното моделиране на данните и заявките: логическата структура на достъпа не се описва чрез низове, проверявани при изпълнение, а чрез типизирани `constexpr` конструкции. Втората е асинхронната архитектура, изградена върху корутини (coroutines), чрез която същият типово ограничен модел се разширява към неблокиращо или псевдо-неблокиращо изпълнение според възможностите на конкретния драйвер.

Разработената библиотека използва статичен модел на данните, изразен в C++ чрез `property<T, Name>`, `relationship<...>` и `table_name_trait<T>`, и типизирано междинно представяне на заявката (`select_query`, `insert_query`, `update_query` и `delete_query`). Този логически слой е независим от конкретната система за съхранение (backend). Материализацията към релационни и нерелационни системи се осъществява чрез специализации на `connector_trait<DB>`, които декларират поддържаните способности и превеждат едно и също междинно представяне към структурирания език за заявки (Structured Query Language, SQL), двоично представяне на JSON (Binary JSON, BSON), Redis команди, Cypher или Cassandra Query Language (CQL) според избраната система за съхранение.

Съществено разширение е асинхронният слой, изграден върху `Task<T>`, `ThreadPool`, `async_db<DB>` и управление на връзките, интегрирано с корутинния модел. Показано е, че единна асинхронна абстракция може да бъде реализирана по два начина: чрез изнасяне на блокиращи операции в обединение от нишки и чрез неблокиращи интерфейси, предоставени от драйверите, които ги поддържат. По този начин работата не описва само логическа абстракция за обектно-релационно картографиране, но и моделът за изпълнение, чрез който тази абстракция остава практически приложима при конкурентни натоварвания. Анализират се както релационните сценарии (SQLite, PostgreSQL, MySQL), така и нерелационните (MongoDB, Redis, Neo4j, Cassandra).

Верификацията се основава на единични (unit) и интеграционни тестове (271 теста), а емпиричната оценка използва набор от измервания, който сравнява синхронния и

асинхронния режим. Резултатите показват ускорение до $23\times$ при сценарии, в които фазата на изчакване доминира над локалната изчислителна работа.

Основните приноси на дипломната работа са четири. Първо, представен е практически приложим модел за статично типизирана библиотека за обектно-релационно картографиране, която съчетава типова безопасност и независимост от системата за съхранение. Второ, показано е как описанието на данните в C++ може да служи като първичен източник на логическата схема за различни типове бази данни. Трето, изградена е асинхронна архитектура върху корутини, която разширява модела на конектора без компромис с компилационните ограничения. Четвърто, формализиран е договор на конектора, чрез който библиотеката може да се надгражда с нови системи за съхранение и асинхронни възможности без промяна в публичния интерфейс на заявките.

Ключови думи: C++23, обектно-релационно картографиране по време на компилация, статична типова безопасност, корутини, асинхронно изпълнение, `connector_trait`, релационни и нерелационни бази данни, емпирична оценка.

I. УВОД

1.1. Постановка на проблема

Достъпът до данни остава едно от местата, в които разминаването между езика на приложението и модела на съхранение поражда най-много структурни грешки. В традиционните слоеве за достъп до бази данни заявките се описват като низове, които се проверяват едва при изпълнение (runtime). Това означава, че неправилни имена на колони, несъвместими типове, невалидни конструкции за свързване (JOIN) или неправилно подадени параметри се откриват късно — често върху реална база данни и след допълнителни ръчни тестове. Същевременно смяната на системата за съхранение (backend) от релационна към документна, графова или ключ-стойност (key-value) обикновено изисква пълно пренаписване на слоя за достъп до данни.

Обектно-релационното картографиране (Object-Relational Mapping, ORM) е класическият подход за свързване на обектния модел на приложението с персистентното съхранение [2]. Повечето реализации обаче остават силно зависими от механизми, действащи по време на изпълнение: текстови заявки на структурирания език за заявки (Structured Query Language, SQL), кодогенерация, анотации, макроси или динамично описвани схеми. В контекста на C++ това положение е особено противоречиво. Езикът предлага мощна шаблонна система, изчислимост по време на компилация (compile-time) чрез `constexpr` и концепции (concepts), но в библиотеките за достъп до данни тези механизми често не се използват в пълна степен. Настоящата работа изследва доколко е възможно компилаторът да поеме ролята на първи валидатор на структурата на заявката и на съвместимостта с конкретната система за съхранение.

Съществена особеност на разработваната библиотека е, че въпреки името „обектно-релационно картографиране“, нейната архитектура не е ограничена само до релационния модел. Напротив, тя използва един и същ C++ модел на данните и едно и също типизирано междинно представяне на заявката (query intermediate representation, query IR) като логически слой над релационни и нерелационни системи. Това поставя втория изследователски въпрос: кои части на ORM абстракцията са универсални и кои неизбежно трябва да се ограничават чрез договор, базиран на способности (capability-based contract).

Третият проблемен пласт е свързан с начина на изпълнение. Дори когато

логическият модел и междинното представяне са типове коректни, практическата стойност на една съвременна библиотека за достъп до данни зависи и от това дали тя интегрира асинхронно изпълнение, без да разруши типовите гаранции и без принудително преминаване към стил, основан на обратни извиквания (callbacks). Така дипломната работа се организира около две допълващи се оси: моделиране по време на компилация и асинхронна архитектура на изпълнението, изградена върху корутини (coroutines).

1.2. Цел и задачи на разработката

Основната цел на дипломната работа е да представи проектирането и реализацията на библиотека за обектно-релационно картографиране по време на компилация за C++, която:

1. моделира данните и заявките чрез статично типизирани C++ конструкции;
2. позволява изграждане на SELECT, INSERT, UPDATE и DELETE заявки като `constexpr` обекти;
3. отделя логическата структура на заявката от специфичната за системата за съхранение материализация;
4. поддържа релационни и нерелационни конектори чрез `connector_trait<DB>`;
5. използва механизъм на способностите за ограничаване на неподдържани операции още при компилация;
6. демонстрира как C++ моделът определя логическата схема и физическата структура на данните при различни системи за съхранение;
7. реализира асинхронен слой, основан на корутини, който надгражда синхронния модел по формален и типове безопасен начин;
8. формализира договор за добавяне на нови конектори, включително такива с неблокираща поддръжка от страна на драйвера;
9. валидира архитектурата чрез систематични единични (unit) и интеграционни тестове и емпирична оценка.

За постигането на тази цел работата решава следните задачи: анализ на съществуващите решения; дефиниране на статичен модел на обектите; изграждане на междинно представяне на заявката; проектиране на конекторния слой и маркерите за способности; изграждане на асинхронен слой, основан на корутини; реализация на конектори за релационни и нерелационни бази данни; изследване на миграции, подготвени заявки, нишкова безопасност и асинхронно управление на връзките; формулиране на критерии за разширяемост; изграждане на методология за емпирична оценка на синхронния и асинхронния режим.

1.3. Обхват и ограничения

Работата разглежда библиотеката като слой за моделиране, изграждане и изпълнение на заявки. В обхвата ѝ попадат: моделът на обектите, междинното представяне на заявката, механизмът на заместителите (placeholders), подготвените заявки, миграционният модел, архитектурата на конекторния слой, ограниченията чрез маркери за способности, асинхронният слой на корутините и конкретните реализации на конектори. Емпиричната оценка покрива релационни (SQLite, PostgreSQL, MySQL), документни (MongoDB) и ключ-стойност (Redis) системи; ширококолонният (Cassandra) и графовият (Neo4j) сценарий се разглеждат само в обобщителен план поради ограниченията на обема.

Извън непосредствения обхват остават оптимизации на производствено равнище като разпределено управление на връзките, автоматизирани миграционни стратегии за всички системи за съхранение, оценка върху пълна матрица от производствени среди и пълно използване на бъдещата функционалност за отражение (reflection) на C++26. Тези направления са разгледани като перспектива в Глава V.

1.4. Структура на изложението

Дипломната работа е организирана по следния начин. Глава II разглежда съществуващите C++ решения за достъп до бази данни и подходите за обектно-релационно картографиране и позиционира настоящата разработка спрямо тях. Глава III представя проектните решения: теоретичната основа на обектно-релационното картографиране, моделите за съхранение, слоестата организация на библиотеката, обектния и заявковия слой, формалния договор на конектора, механизма на способностите и теорията на

корутините и асинхронния дизайн. Глава IV описва програмната реализация: асинхронния изпълнителен слой, конкретните конектори за основните системи за съхранение и подробната верификация чрез единични, интеграционни и емпирични тестове. Глава V обобщава приносите, ограниченията и направленията за бъдещо развитие.

II. ОБЗОР НА СЪЩЕСТВУВАЩИ РЕШЕНИЯ

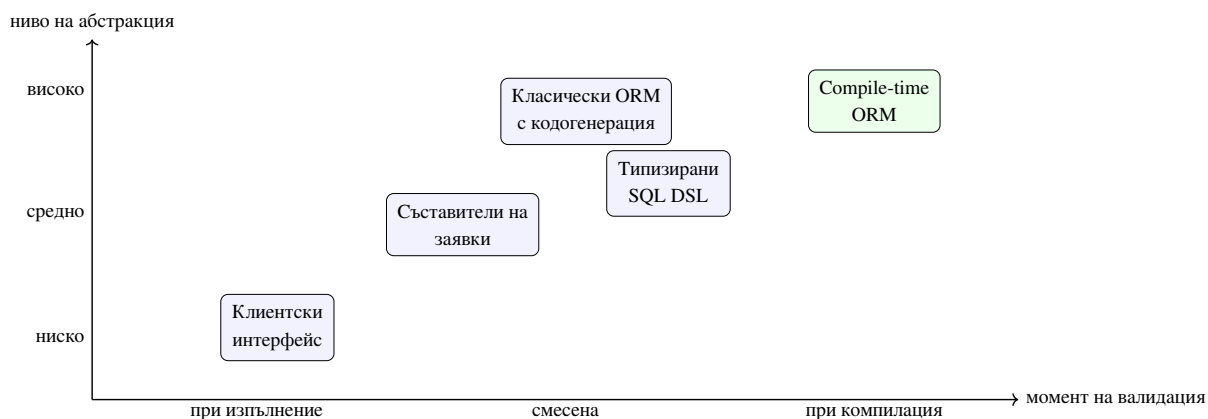
Прегледът на съществуващите решения е необходим, за да се прецени дали предложената архитектура действително предоставя нова стойност, или само преформулира вече познати практики. При C++ библиотеките за достъп до данни могат да бъдат разграничени две основни оси и една допълнителна изпълнителна перспектива: ниво на абстракция, момент на изграждане и валидиране на заявката, и модел на изпълнение. Нивото на абстракция варира от почти пряко използване на C-интерфейс на драйвера до пълно поведение на обектно-релационно картографиране (ORM). Моментът на валидиране може да бъде по време на изпълнение (runtime), смесен или по време на компилация (compile-time). Моделът на изпълнение варира от блокиращи извиквания към системата за съхранение (backend) през специфични за конкретната система асинхронни крайни автомати до по-високи слоеве, които правят опит да ги уеднаквят.

2.1. Класификация на подходите

В най-ниския слой стоят клиентските библиотеки на драйверите като SQLite C API [3], libpq [4] и MySQL Connector/C [5]. Те предоставят максимален контрол върху заявките, параметрите и резултатите, но не предлагат логическа абстракция. Приложението работи пряко с низове на структурирания език за заявки (Structured Query Language, SQL), индекси на колони, позиции на параметрите и специфични за системата за съхранение типове.

Следващата група са съставителите на заявки по време на изпълнение (runtime query builders). Те подобряват четимостта на заявките и намаляват част от шаблонния код (boilerplate), но продължават да разчитат на изграждане на текстово представяне по време на изпълнение. Типовата безопасност е частична и обикновено спира до границата на извикването. Структурната валидност на заявката остава проблем на изпълнението.

Най-високото ниво на абстракция е подходът на обектно-релационното картографиране. При него C++ класовете се асоциират с таблици, колони и връзки, а библиотеката генерира съответните заявки. Предимството е намаляване на шаблонния код; недостатъкът е, че често се въвеждат външни генератори, макроси, анотации и непрозрачни метаданни по време на изпълнение, които затрудняват проследимостта между C++ кода и реалната заявка.



Фигура 1. Позициониране на основните семейства решения според абстракция и момент на валидиране

От Фигура 1 се вижда, че подходът на обектно-релационното картографиране по време на компилация не е „още една ORM библиотека“. Той стои в горния десен квадрант, в който високото ниво на абстракция се съчетава с ранна структурна проверка. Тази комбинация е рядка, защото изисква по-голяма архитектурна дисциплина в самата библиотека.

2.2. Критерии за сравнение

За да бъде анализът академично пълен, не е достатъчно да се каже, че една библиотека е „по-удобна“, а друга — „по-ниско ниво“. В настоящата работа се използват шест сравнителни критерия: ниво на логическа абстракция; степен на статична валидация; преносимост между системи за съхранение; сложност на инструменталната верига (toolchain); прозрачност на грешките и изпълнението; модел на изпълнение и пригодност за асинхронен достъп. Тези критерии са пряко свързани с целите на дипломната работа — ако предложената библиотека трябва да бъде едновременно изразителна и формално проверяема, тя трябва да превъзхожда алтернативите не само по удобство, а по баланса между тези шест измерения.

2.3. Анализ на представителните решения

2.3.1. Клиентски библиотеки на драйверите

SQLite [6], libpq и MySQL Connector/C са представителни за подхода, при който приложението конструира и подава заявките пряко към драйвера. Това е

най-универсалният и често най-бързият вариант, но поставя цялата отговорност за коректността върху приложния код. Разработчикът ръчно поддържа съответствие между имената на полетата, поредността на параметрите и реалната схема на базата данни. При смяна на системата за съхранение се променя не само диалектът на заявката, но и целият интерфейс за свързване, подготвяне (preparation), свързване на параметри (parameter binding) и четене на резултати. Подходът е особено проблематичен, когато един и същ модел на областта трябва да работи последователно с повече от една система за съхранение — приложението започва да натрупва условни пътища, специфични за системата за съхранение низове и дублирана логика за материализиране на резултатите.

Съществена част от съвременния контекст е, че клиентските библиотеки на драйверите не са само синхронни. `libpq` предоставя крайно автоматно представяне за асинхронно изпращане и получаване на заявки [7]; `MySQL Connector/C` предлага двойки от вида `_start/_cont` за неблокиращи операции [8]; `hiredis` има слой, основан на обратни извиквания [9], а драйверът за `Cassandra` работи чрез механизъм с бъдещи стойности (futures) и обратни извиквания [10]. При `MongoDB` акцентът остава върху безопасно използване на пул от връзки в многонишков контекст чрез `libmongoc` [11], а при `libneo4j-client` преобладава синхронният модел заявка-отговор [12]. Тази нееднородност прави необходим междинен асинхронен слой, който едновременно е честен спрямо различията на системите за съхранение и е достатъчно единен за публичния програмен интерфейс.

2.3.2. Съставители на заявки и типизирани SQL DSL

Библиотеки като `SOCI` [13] търсят баланс между четимост и контрол. Те предлагат по-удобен `C++` интерфейс за изграждане на заявки, но в много случаи продължават да разчитат на описание по време на изпълнение и последващо изобразяване към `SQL`. Известна част от структурата на заявката е по-добре капсулирана, но смяната на модел на съхранение, независима от системата за съхранение, остава ограничена. `sqlpp11` [14] стои между традиционния съставител и по-строгий статичен дизайн: подобни библиотеки използват шаблони и типове, за да направят част от `SQL` структурата видима за компилатора. Същевременно типизираните `SQL DSL` обикновено остават дълбоко свързани с релационния модел — те предполагат свят, в който таблици, колони, свързвания (joins) и агрегати са централната метафора, и са по-трудно преносими към документни, графови или ключ-стойност системи.

2.3.3. Класически рамки за обектно-реляционно картографиране

ODB [15] е характерен пример за рамка, която предлага богато картографиране между класове и реляционни таблици чрез генериране на код и външни инструменти. Това улеснява потребителя на ниво употреба, но измества сложността към процеса на компилация и към автоматично генерираните артефакти. Получава се силна зависимост между модела на данните, допълнителната стъпка от инструменталната верига и конкретните реляционни системи за съхранение. В класическите ORM рамки често се наблюдава и друго напрежение: колкото по-високо е удобството на ниво приложение, толкова по-непрозрачен става пътят до реалната заявка и до характеристиките на производителност. За приложения, които трябва да работят с документни, графови или ключ-стойност системи, класическият ORM модел показва естествени ограничения — част от концепциите като свързване, външни ключове и строго таблична схема нямат пряк еквивалент във всички модели за съхранение.

2.4. Обобщителна сравнителна таблица

Таблица 1 обединява разгледаните семейства решения по четири основни критерия. Тя играе ролята на компактен сравнителен инструмент, който замества по-обширните и частично припокриващи се таблици от ранните версии на анализа.

Таблица 1. Съпоставка на основните подходи за достъп до данни в C++

Подход	Ниво на абстракция	Валидация	Предимство	Основен недостатък
Клиентски интерфейс	ниско	при изпълнение	пълен контрол и близост до драйвера	силна обвързаност с конкретна система за съхранение
Съставител на заявки	средно	предимно при изпълнение	по-четим интерфейс и по-малко шаблонен код	заявката пак се материализира като низ
Типизиран SQL DSL	средно към високо	частично при компилация	по-добра статична структура за SQL	остава ограничен до релационния модел
Класически ORM	високо	при изпълнение или чрез кодогенерация	удобно картографиране за релационни системи	сложна инструментална верига и непрозрачност
Compile-time ORM	високо	при компилация за структурата	ранно откриване на структурни грешки и формален договор на конектора	по-висока сложност на шаблонния слой

2.5. Позициониране на настоящата разработка

Предложената библиотека се позиционира като решение за обектно-релационно картографиране по време на компилация, което съчетава силна статична типизация, отсъствие на външна кодогенерация, модел на конекторите за пренасяне на логическата абстракция към различни системи за съхранение и асинхронен слой, основан на корутини. Спрямо прекия интерфейс на драйвера тя намалява дублирането на логиката и риска от късно открити грешки. Спрямо съставителите на заявки тя премества структурната валидация към компилатора. Спрямо класическите ORM рамки тя избягва външни генератори и поставя ясен акцент върху договора на конектора. Спрямо специфичните за системата за съхранение асинхронни интерфейси тя предлага единен корутинен модел, без да твърди, че всички драйвери имат еднакви неблокиращи възможности.

Ключовата разлика обаче е по-дълбока. Настоящата разработка не разглежда статичните техники като средство за „по-умен съставител на SQL“, а като начин за формализиране на логическия модел на достъпа до данни. Аналогично, тя не разглежда асинхронността като декоративна обвивка, а като отделен архитектурен проблем със собствени ограничения, правила за жизнен цикъл и специфични пътища за изпълнение спрямо системата за съхранение. Точно това позволява следващите глави да преминат от проектните решения и теорията на корутините към реализацията, конкретните конектори и многобройните сценарии за съхранение.

Сравнителният анализ води до четири основни извода. Първо, преките интерфейси на драйверите остават незаменими като отправна точка за контрол и производителност, но не предлагат удовлетворително ниво на логическа абстракция. Второ, съставителите на заявки и типизираните SQL DSL подобряват работата със SQL, но не решават напълно проблема за логически слой, независим от системата за съхранение. Трето, класическите ORM рамки предлагат висока абстракция, но често за сметка на сложна инструментална верига и ограничена пригодност извън релационния свят. Четвърто, асинхронните клиентски интерфейси, предоставени от драйверите, са ценни, но показват толкова различни модели на изпълнение, че сами по себе си не осигуряват единна потребителска абстракция. Именно това позициониране подчертава необходимостта от ясно проектен слой над тях, на който е посветена следващата глава.

III. ПРОЕКТИРАНЕ НА БИБЛИОТЕКАТА

Настоящата глава представя проектните решения, върху които стъпва библиотеката за обектно-релационно картографиране (Object-Relational Mapping, ORM). Излагането върви от теоретичната основа на статичното моделиране, през слоестата организация и логическите слоеве (обекти, заявки, конектори), до асинхронната подсистема, изградена върху корутини (coroutines). Целта е читателят да види архитектурата като система от типове, формални договори (contracts) и проверими ограничения, а не като отделни езикови похвати.

3.1. Теоретична основа на обектно-релационното картографиране

Достъпът до данни е една от областите, в които разминаването между езика на приложението и модела на съхранение поражда най-много структурни грешки. Когато заявката се описва като низ, който се проверява едва при изпълнение, неправилните имена на полета, несъвместимите типове и невалидните клаузи се откриват късно. Класическият подход на обектно-релационното картографиране [2] свързва обектния модел на приложението с персистентното съхранение, но традиционните реализации остават силно зависими от изпълнителни механизми: текстови заявки, кодогенерация, анотации или динамично описани схеми.

В контекста на C++ това положение е особено противоречиво. Езикът предоставя мощна шаблонна (template) система, изчислимост по време на компилация (compile-time) чрез `constexpr` и концепции (concepts), но в библиотеките за достъп до данни тези механизми обикновено не се използват в пълна степен. Настоящата разработка изследва доколко самият компилатор може да поеме ролята на първи валидатор на структурата на заявката и на съвместимостта със системата за съхранение (backend).

Тук терминът обектно-релационно картографиране не се разбира тясно като техника, ограничена до релационни бази данни. Той обозначава архитектурен подход, при който моделът на данните, формата на заявките и част от ограниченията върху операциите се описват чрез типовата система на езика, а не чрез конфигурация по време на изпълнение (runtime). Това позволява една и съща логическа структура да бъде пренесена към релационни, документни, ключ-стойност (key-value), графови и ширококолонни (wide-column) системи, без различията им да се скриват или подменят.



Фигура 2. Таксономия на логическия модел, междинното представяне на заявката и семействата системи за съхранение

Фигура 2 подчертава централното теоретично допускане на библиотеката: общото между различните системи за съхранение не е техният синтаксис, а логическият модел на данните и операциите. Затова валидната абстракция трябва да бъде разположена над структурирания език за заявки (Structured Query Language, SQL), двоичното разширение на JSON (Binary JSON, BSON), Redis командите, Cypher и Cassandra Query Language (CQL). Тя трябва да работи на равнището на полета, предикати, проекции, връзки и допустими операции.

Съществената особеност на подхода е, че моделирането по време на компилация не означава отсъствие на данни по време на изпълнение. Стойностите на параметрите се подават именно при изпълнение, но структурата на заявката е фиксирана и проверена още преди това. Получава се ясно разграничение между *логическа форма* и *стойности по време на изпълнение*, което е ключово за тезата на дипломната работа, защото позволява едновременно статични гаранции и работа с реални системи за съхранение.

3.2. Модели за съхранение

Библиотеката работи над пет основни модела за съхранение. Релационният модел организира данните в таблици, редове и колони и предоставя декларативен език за

заявки със ясно дефинирани ограничения на целостта. Документният модел работи с колекции от документи, чиито полета могат да бъдат вложени и да варират по структура. Ключ-стойност моделът осигурява бърз достъп около първичен идентификатор, но има силно ограничени възможности за заявки върху предикати. Графовият модел представя данните като върхове, ребра и свойства, превръщайки връзките в първокласни обекти. Ширококолонният модел, представен от Cassandra, използва разпределени таблици с разделящи (partition) и подреждащи (clustering) ключове, при които правилната заявка следва физическата схема на разпределение.

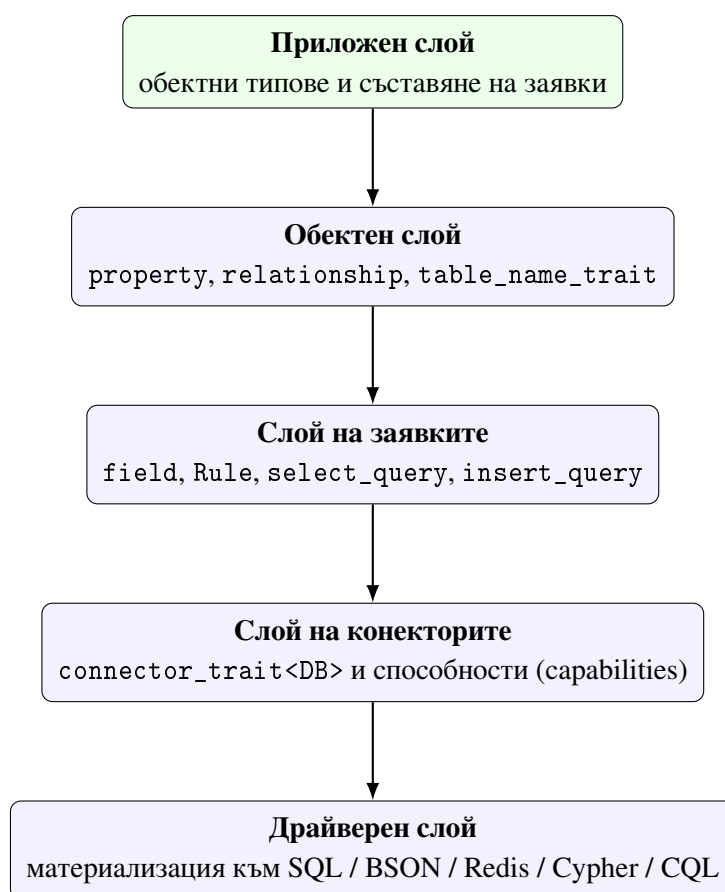
Таблица 2. Съпоставка между основните модели за съхранение

Модел	Единица за съхранение	Типични връзки	Основно ограничение
Релационен	таблица / ред	външен ключ, свързване (JOIN)	необходимост от съгласувана схема и силна типова дисциплина
Документен	документ	вграждане или препратка	не всяка релационна операция е естествена или евтина
Ключ-стойност	ключ + стойност	справка по ключ, хеш-полета	силно ограничени заявки по предикати извън ключовия достъп
Графов	върх + ребро	връзки за обхождане	специфичен език за заявки и графова семантика
Ширококолонен разделен ред		ключово ориентирани зависимости	заявките трябва да следват логиката на разделящия ключ

От Таблица 2 се вижда, че унифицирана абстракция е възможна само на равнището на логическия модел, полето, връзката и базовите операции. Отвъд тази граница започват различията: свързване няма универсален смисъл в Redis; вграждането няма идентична цена в релационна таблица и в документна колекция; обхождане на граф не се свежда до проста селекция; ограниченията, наложени от разделящия ключ в Cassandra, не могат да бъдат прикрити като обикновени правила за филтриране. Архитектурно правилният подход не е тези различия да се скриват, а да се направят явни и формално контролирани.

3.3. Архитектурни цели и слоеста организация

Архитектурата на библиотеката е проектирана при едновременното действие на няколко ограничения. Първо, публичният програмен интерфейс трябва да остане в рамките на идиоматичен C++, без отделен специализиран език (Domain-Specific Language, DSL) и без отражение (reflection) по време на изпълнение. Второ, структурата на заявката трябва да бъде представима като `constexpr` обект, така че значима част от валидирането да се извършва от компилатора. Трето, слой на конекторите трябва да бъде разширяем без наследяване от общ виртуален интерфейс, тъй като такъв интерфейс би скрил различията между системите за съхранение и би изместил проверките към изпълнение. Четвърто, библиотеката трябва да поддържа релационни и нерелационни конектори без фалшиво уеднаквяване — общият програмен интерфейс и ограниченията на конкретната система за съхранение трябва да съществуват едновременно и проверимо.



Фигура 3. Слоеста архитектура на библиотеката

Фигура 3 показва последователната вертикална организация. Приложният код не общува пряко с драйвера, а работи с C++ типове и съставители на заявки (query

builders). Драйверът от своя страна не познава семантиката на областта, а получава вече материализираната форма, генерирана от слоя на конекторите. Когато потребителят смени SQLiteDB с MongoDB, междинното представяне на заявката остава логически същото, но конекторът го тълкува като изпълнение върху филтър и проекция вместо изобразяване към SQL. Това не означава, че всяко междинно представяне е допустимо върху всяка система за съхранение; означава, че *формата* на абстракцията е обща, а допустимостта се определя формално чрез механизма на способностите.

3.4. Обектен слой

Обектният слой е изграден около три централни конструкции: `property<T, Name>`, `relationship<...>` и `table_name_trait<T>`. Те имат двойна роля — представят C++ модела на областта и същевременно описват логическата структура на персистентните данни. Така в библиотеката не съществува отделен регистър от метаданни по време на изпълнение; типовата система съдържа достатъчно информация, за да бъдат извлечени имената на полетата, типовете, връзките и целевият контейнер за съхранение.

Имената на логическите контейнери чрез `table_name_trait<T>` играят обобщена роля. В SQLite, PostgreSQL и MySQL те означават таблица; в MongoDB — колекция; в Redis — ключов префикс; в Neo4j — етикет (label). Единен логически интерфейс адресира физически различни структури, а авторът на конектора извлича схемата директно от типовете, а не от външен регистър.

```
1 template <typename T, detail::string_literal Name>
2 struct property
3 {
4     using value_type = T;
5
6     [[nodiscard]] static constexpr std::string_view column_name() noexcept
7     {
8         return Name.view();
9     }
10
11     T value{};
12 };
13
14 enum class store_as { reference, embed };
15
16 template <store_as Strategy, typename T, detail::string_literal Name>
17 struct relationship
18 {
19     static constexpr store_as strategy = Strategy;
```

```

20     using related_type = T;
21 };

```

Listing 1: Извадка от `property` и `relationship` в обектния слой

Listing 1 показва, че обектният слой е минималистичен по форма, но достатъчно богат по смисъл. `property` не е просто контейнер за стойност, а носи и компилационно име на колоната. `relationship` не е обикновен член, а фиксира стратегията за материализация като част от типа: `store_as::reference` обозначава логическа външна препратка (foreign key, справка, ребро в граф), а `store_as::embed` обозначава физическо включване в родителската структура (вложен документ, денормализирани данни). Връзката се описва веднъж на логическо ниво, а интерпретацията остава задача на конкретния конектор.

3.5. Слой на заявките

Вместо заявката да се описва пряко като SQL или друго текстово представяне, библиотеката използва типизирано междинно представяне на заявката (query intermediate representation, query IR). Неговите елементи са `field`, `Rule`, `Placeholder` и обектите `select_query`, `insert_query`, `update_query` и `delete_query`. Този модел има две предимства. Първо, структурата на заявката може да бъде анализирана по време на компилация и да участва в `static_assert` проверки. Второ, едно и също междинно представяне може да бъде преведено към различни езици и протоколи за съхранение.

Концептуално междинното представяне играе ролята на междинен език между модела на областта и слоя за изпълнение. Това е аналогично на междинните представяния в компилаторите: потребителят работи с високо ниво на абстракция, а конкретната система за съхранение получава специализирана форма, оптимизирана за собствения си модел.

```

1  template <typename Response, typename Joins, typename Wheres,
2          typename Limits, typename Groups, typename Orders>
3  struct select_query : select_query_tag
4  {
5      using response_type = Response;
6      using row_type = projected_type<Response>;
7
8      template <typename RuleType>
9          requires is_rule<RuleType>
10     [[nodiscard]] constexpr auto where(RuleType r) const
11     {
12         using NewWheres = detail::append_type_t<Wheres, RuleType>;
13         return select_query<Response, Joins, NewWheres, Limits, Groups, Orders>(<

```

```

14         response_, joins_,
15         detail::tuple_cat(wheres_, detail::orm_tuple<RuleType>(r)),
16         limits_, groups_, orders_);
17     }
18 };

```

Listing 2: Извадка от `select_query` и съставяне на правила

Listing 2 илюстрира важно теоретично свойство: съставителят на заявки не променя низ, а изгражда нови типизирани обекти. Последователността `select(...).where(...).group_by(...)` представлява конструиране на нов тип, а не серия от странични ефекти върху текстова форма.

Полето в израза на заявката се представя чрез `field<&T::m>` — това е `constexpr` обвивка над указател към член, която носи типа на контейнера, типа на стойността и името на колоната. Операторите върху `field` създават типизирани правила (`Rule`). Изразът `field<&User::id> == Placeholder<int>{}` не е низ, а статично описание на предикат.

Механизмът на заместителите има две форми. `Placeholder<T>` моделира анонимни параметри, които се консумират позиционно от аргументите на `execute()`. `orm::ph<T, std::placeholders::_N>` моделира индексирани параметри, при които една стойност по време на изпълнение може да се използва на повече от едно място. Тази възможност е особено полезна за системи за съхранение като SQLite, PostgreSQL и MySQL, при които позицията на заместителя е отделна част от логиката на материализация. Тъй като веригата на съставителя е съвместима с `constexpr`, значима част от формата на заявката може да бъде изградена и проверена още при компилация (Listing 3).

```

1 using namespace orm::literals;
2 static constexpr auto get_active =
3     orm::select(orm::field<&User::id>, orm::field<&User::name>)
4         .where(orm::field<&User::score> > 0.0)
5         .where(orm::field<&User::id> == orm::Placeholder<int>{})
6         .limit(10_per_page & 2_page);

```

Listing 3: Заявка декларирана като константа, изчислима при компилация

3.6. Слой на конекторите и формален договор

Конекторът се описва чрез тип `DB` и специализация `connector_trait<DB>`. Това е структурен, а не наследствен договор: няма общ виртуален интерфейс `Connector`, защото

различните системи за съхранение имат различни възможности и ограничения, които трябва да бъдат видими по време на компилация. Виртуалната йерархия би изравнила твърде много различия и би прехвърлила част от проверките към изпълнение.

Минималният работещ конектор трябва да предостави `wire_type`, `cursor_type` и подходящи претоварвания на `execute(...)`. Допълнителните възможности се обозначават чрез маркери за способности (capability tags). По този начин договорът остава едновременно строг и разширяем; компилаторът вижда специализацията директно и може да провери дали тя удовлетворява концепцията `is_connector`.

```
1 template <typename DB>
2 concept is_connector = requires {
3     typename connector_trait<DB>::template wire_type<int>::type;
4     typename connector_trait<DB>::cursor_type;
5 };
6
7 template <typename DB, typename Cap>
8 inline constexpr bool has_capability =
9     detail::capability_check<DB, Cap>::value;
```

Listing 4: Извадка от формалния договор на конектора

Listing 4 показва, че договорът е структурен, а не наследствен. Това е решаващ архитектурен избор: ако съществуваше виртуален интерфейс с диспечеризация по време на изпълнение, част от типово проверимата информация би била изгубена. Вместо това библиотеката предпочита статична структура, при която изискванията са видими както за компилатора, така и за автора на конектора.

Таблица 3. Задължителни и опционални елементи на конектора

Елемент	Роля
DB тип	обвивка около връзка, сесия или имитиращ (mock) обект
<code>connector_trait<DB></code>	централна специализация на договора
шаблон <code>wire_type<T></code>	превръща C++ тип в представяне, използвано от драйвера (wire representation)
<code>cursor_type</code>	описва обхождането и жизнения цикъл на резултатите
<code>execute(...)</code>	материализира междинното представяне към действие на системата за съхранение
Маркери за способности	ограничават допустимите операции и политики на жизнения цикъл
<code>begin/commit/rollback</code>	необходими при поддръжка на транзакции
<code>ddl_for(...)</code>	необходимо при интеграция с миграция на схемата

Имената на конекторите следват ясна конвенция: SQLiteDB, PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB и CassandraDB обозначават локални или mock-подобни обвивки, докато реалните варианти, които ползват жив (live) драйвер, носят суфикс LiveDB. Това не е стилистичен избор, а част от яснотата на договора: още от името личи дали типът е тестов адаптер, локална обвивка или система за съхранение, обвързана с реален драйвер.

3.7. Механизъм на способностите

Маркерите за способности са основният начин библиотеката да различава конекторите по допустими операции. Ако `connector_trait<DB>` не декларира `supports_joins`, опитът за заявка със свързване (JOIN) е архитектурно неправилен и трябва да бъде отхвърлен още при компилация. Същото важи за транзакциите, за агрегацията и за конкурентното изпълнение. По този начин коректността не зависи единствено от тестове — част от договора се проверява пряко от компилатора.

Таблица 4. Основни маркери за способности и тяхното значение

Способност	Архитектурно значение
<code>supports_joins</code>	разрешава заявки със свързване за дадената система за съхранение
<code>supports_transactions</code>	позволява <code>transaction_guard</code> и куки за <code>begin/commit/rollback</code>
<code>supports_aggregation</code>	обозначава поддръжка на агрегиращи операции в слоя на заявките
<code>supports_embedding</code>	декларира естествена поддръжка на вградени структури
<code>supports_upsert</code>	маркира системи за съхранение, при които вмъкване-или-обновяване (<code>upsert</code>) е част от естествения договор
<code>supports_bulk_insert</code>	обозначава наличие на ефективен път за масово записване
<code>supports_concurrent_execute</code>	необходимо за <code>connection_pool<DB, N></code>
<code>supports_async</code>	задължително за интеграция чрез неблокиращите интерфейси на драйвера в асинхронния слой

Класът `db<DB>` е публичното фасадно ниво. Той обвива препратка към конкретна система за съхранение и предоставя унифициран интерфейс за изпълнение. Същевременно той е архитектурен филтър: точно тук статичните проверки за способности и съвместимост се срещат с конкретното междинно представяне на заявката.

```

1  template <typename DB>
2      requires is_connector<DB>
3  class db
4  {
5  public:
6      template <typename Query>
7      auto operator<<(Query q)
8      {
9          if constexpr (is_select_query<Query>)
10         {
11             if constexpr (decltype(q.join_clauses())::size > 0)
12             {
13                 static_assert(has_capability<DB, cap::supports_joins>,
14                               "orm::db: this connector does not support JOIN operations.");

```

```

15         }
16     }
17     return connector_trait<DB>::execute(*conn_, std::move(q));
18 }
19 };

```

Listing 5: Проверка на възможностите във фасадата db<DB>

Listing 5 показва защо db<DB> не е тънка обвивка, а формалното място, на което структурата на заявката и информацията за способностите на конектора се срещат. Именно тук статичната архитектура се превръща в реално ограничение върху употребата на интерфейса. Аналогична логика прилагат и помощниците `connection_pool<DB, N>`, `thread_local_db<DB>` и `transaction_guard<DB>`: всеки от тях изисква декларацията на съответната способност в `connector_trait<DB>`, без която компилацията не преминава.

Особено важно е, че проверката на способностите включва и негативни декларации: ако MongoDB или Redis не поддържат свързване и транзакции, отсъствието на съответните маркери също е част от договора и се валидира от тестовете на конкретните конектори. Това различава нашата архитектура от наследствените йерархии, в които „наследниците“ са длъжни да реализират цялостен интерфейс независимо дали той има смисъл за тяхната семантика.

3.8. Прототипен слой за миграции

Архитектурата включва и прототипен слой за миграция на схемата чрез `migrate<DB>`, `live_schema`, `entity_meta` и операции на езика за описание на данни (Data Definition Language, DDL) като `create_table_op`, `add_column_op`, `drop_column_op` и `alter_column_type_op`. Този слой не е универсално завършен за всяка система за съхранение, но архитектурно показва, че библиотеката не се ограничава до изпълнение на заявки. При достатъчно богата специализация на `connector_trait<DB>` обектното описание участва и в развитието на схемата. Това променя смисъла на слоя за обектно-релационно картографиране: той се превръща не само в интерфейс към данните, но и в потенциален източник на инструменти, осведомени за схемата (schema-aware tooling). Конкретното изпълнение на тази подсистема и подкрепящите я тестове са описани в Глава IV.

3.9. Теория на корутините и асинхронен дизайн

Втората основна архитектурна ос на разработката е асинхронното изпълнение. Корутините (coroutines) са езиков механизъм за описване на прекъсваеми изчисления, при които изпълнението може да бъде спряно и по-късно възобновено без загуба на локалното състояние. В стандарта C++20 те влизат в основния език [16, 17, 18], което позволява асинхронният слой да се изгражда върху статично дефинирани правила за типове, жизнен цикъл и поток на управление, вместо върху ad hoc обратни извиквания (callbacks).

3.9.1. Корутини спрямо нишки и процеси

Процесът е защитен адресен контекст на операционната система, нишката е планирана от ядрото последователност от инструкции, а корутината е кооперативна единица за изпълнение, чието спиране и възобновяване се управляват от програмния модел, а не пряко от планировчика на ядрото. Това разграничение е съществено, защото библиотеката комбинира корутини и обединение от нишки (thread pool) без да приравнява двете абстракции. Нишката е скъп ресурс на ядрото със собствен стек и синхронизационни примитиви; корутината е значително по-лека логическа единица, която пази само състоянието, необходимо за конкретното изчисление.

Таблица 5. Съпоставка между основните изпълнителни единици

Единица	Същност	Значение за библиотеката
Процес	Изолиран адресен контекст с ресурси на операционната система	Не е пряка единица на архитектурата, но определя средата за изпълнение
Нишка	Поток на изпълнение, планиран от ядрото	Използва се от обединението от нишки за изнасяне на блокиращи операции
Корутина	Кооперативно прекъсваемо изчисление със запазено локално състояние	Осигурява формалния модел за Task<T> и за асинхронната композиция

3.9.2. Awaitable договор и преобразуване в крайно автоматно представяне

Ключовите езикови оператори `co_await`, `co_return` и `co_yield` маркират, че функцията следва корутинна семантика. Най-важен е `co_await`: той описва точката, в която текущото изчисление отстъпва изпълнението, докато външна операция — база данни или планиране към обединението от нишки — стане готова. Така синтаксисът на потребителя остава линеен, докато моделът на изпълнение е асинхронен.

Обектът, който се изчаква, участва в тристъпков договор. `await_ready()` определя дали изобщо е необходимо спиране; `await_suspend()` получава манипулатор на корутината и решава как ще се организира възобновяването; `await_resume()` връща резултата или изнася изключение. Тази тройка обединява разнородни източници на асинхронност — задачи в обединение от нишки, готовност на канал в реактор, обратно извикване от драйвер — под единен синтаксис на `co_await`.

Компиляторът преобразува корутинната функция в крайно автоматно представяне (state machine) с ясно дефинирани точки на спиране, преходи на възобновяване и кадрово (frame) съхранение [17, 18]. Това е особено значимо за тезата на работата: същият статичен инструментариум, който описва заявките, дава възможност и потокът на асинхронно управление да бъде формализиран чрез типове и статични правила, вместо чрез неструктурирани вериги от обратни извиквания.

3.9.3. Task<T> като носител на жизнения цикъл

Основната потребителска абстракция е `Task<T>`. Тя не се изчерпва с „връща бъдещ резултат“ — тя е контейнер за корутинния кадър, обекта на обещанието (promise object), продължението (continuation) и политиката за завършване. Реализацията използва `initial_suspend()` и `final_suspend()` така, че корутината по подразбиране е мързелива (lazy) и предава управлението обратно към продължението при завършване. Така `Task<T>` е едновременно ръкохватка за изпълнение и формален договор за това как се предават резултат, изключение и завършване между корутинни контексти.

От инженерна гледна точка са важни три свойства. Първо, `operator co_await()` свързва текущото продължение към изчакваната задача и позволява естествено вложено съставяне с `co_await`. Второ, `sync_wait()` осигурява мост към некорутинен код, което е необходимо за единичните (unit) тестове и за граничните сценарии, в които приложението

още не работи изцяло в корутинен стил. Трето, `detach()` и `start_detached()` позволяват контролирано поведение „пусни и забрави“, при което кадърът на корутината се освобождава самостоятелно след `final_suspend()`.

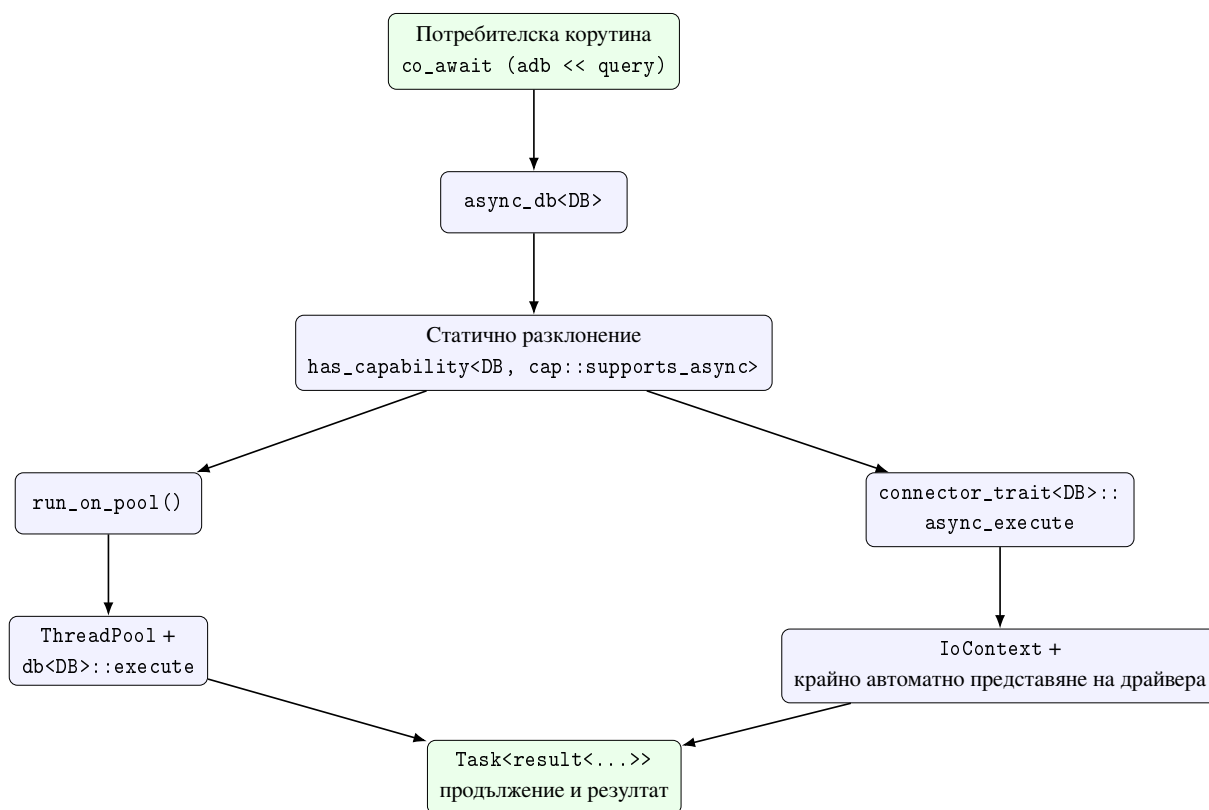
3.9.4. Универсален и асинхронен път през интерфейса на драйвера

Над тази основа стои `async_db<DB>` — асинхронната фасада, която обвива `db<DB>` и запазва синтаксиса на заявката, но връща `Task<result<...>>`. Архитектурно тук се случва най-важният избор: `operator<<` съдържа `if constexpr` разклонение върху `has_capability<DB, cap::supports_async>`. Ако конекторът декларира, че драйверът предоставя асинхронен интерфейс, се извиква `connector_trait<DB>::async_execute()`. В противен случай синхронното изпълнение се обвива в `run_on_pool()`, който публикува заявката към `ThreadPool`, спира извикващата корутина и я възобновява, когато работата приключи.

```
1 template <typename Query>
2 [[nodiscard]] auto operator<<(Query q)
3     -> Task<decltype(std::declval<db<DB>>()) << std::declval<Query>()>>
4 {
5     using ResultType = decltype(std::declval<db<DB>>()) << std::declval<Query>());
6
7     if constexpr (has_capability<DB, cap::supports_async>)
8     {
9         co_return co_await connector_trait<DB>::async_execute(
10             db_.connection(), std::move(q));
11     }
12     else
13     {
14         co_return co_await run_on_pool(
15             *pool_, [this, q = std::move(q)]() mutable -> ResultType {
16                 return db_ << std::move(q);
17             });
18     }
19 }
```

Listing 6: Разклонение на логиката в зависимост от възможностите на съответния конектор в `async_db<DB>`

Listing 6 е централен за цялата асинхронна архитектура. Той показва, че библиотеката не използва регистър по време на изпълнение или диспечеризация на основата на низове, а статична проверка върху способностите на конектора. По този начин потребителският интерфейс остава еднороден, а вътрешният път на изпълнение остава честен спрямо реалните възможности на конкретната система за съхранение.



Фигура 4. Двата пътя на изпълнение на `async_db<DB>`: универсално изнасяне и интеграция през асинхронния интерфейс на драйвера

За драйверите, които предоставят неблокиращи интерфейси, библиотеката не се ограничава до изнасяне в обединение от нишки. Тук влиза в действие `IoContext` — абстракция за реакторен цикъл, която предоставя `run()`, `post()`, `watch_readable(fd)` и `watch_writable(fd)`. Тези обекти не извършват самостоятелно входно-изходна операция; те само спират корутината и я възобновяват, когато съответният файлов описател стане готов за четене или запис. Това разделение между готовност и действие е решаващо: `IoContext` не „крие“ драйвера, а предоставя реакторен интерфейс, върху който клиентската библиотека с неблокиращ интерфейс може да стъпи (например `PQconnectPoll()`, `PQsendQueryParams()`, `PQflush()` и `PQisBusy()` на PostgreSQL).

Така асинхронният път през интерфейса на драйвера не е „по-бързо обединение от нишки“, а качествено различен модел: работната нишка не блокира в очакване на мрежова готовност; корутината се спира, а възобновяването се управлява от наблюдението на реактора. В Глава IV този модел се проявява конкретно при PostgreSQL, докато за SQLite, MySQL, MongoDB и Redis библиотеката използва изнасяне в обединение от нишки като честен и адекватен заместител, защото съответните драйвери нямат неблокираща семантика със същата зрелост.

3.9.5. Координация на връзките и транзакциите при асинхронен достъп

Асинхронната архитектура не приключва с единичното изпълнение на заявка. За реално приложение е необходимо управление на връзките и транзакциите при конкурентен достъп. За тази цел библиотеката предоставя `async_connection_pool<DB, N>`, `async_connection_guard<DB>`, `async_transaction_guard<DB>` и фабричната корутина `async_begin_transaction()`. `async_connection_pool<DB, N>` е продължение на синхронния обединителен слой, интегрирано с корутинния модел: методът `acquire()` връща `Task<async_connection_guard<DB>>` и вътрешно използва `run_on_pool()` и условна променлива, за да изчака свободен слот, без извикващата корутина да бъде изложена на блокиращ интерфейс.

Същата логика се наблюдава и при транзакциите. `async_begin_transaction()` първо изнася `BEGIN` в обединението от нишки и след това връща `async_transaction_guard<DB>`. Методите `commit()` и `rollback()` са `Task<void>` операции, а деструкторът на пазача (`guard`) изпълнява защитно `ROLLBACK`, ако транзакцията не е финализирана изрично. Дизайнът е едновременно удобен и консервативен: успешният път е интегриран с корутинния модел, а пътът за почистване остава детерминистичен.

3.10. Архитектурни инварианти и обобщение

От разгледаните слоеве могат да бъдат формулирани четири инварианта. Първо, приложният код никога не зависи пряко от драйверен интерфейс. Второ, логиката на заявката се формулира в типизирано междинно представяне, а не в специфични за системата за съхранение низове. Трето, разширяването към нова система за съхранение се извършва чрез специализация на признаците (`trait specialisation`) и декларация на способности, а не чрез промяна на самия интерфейс за заявки. Четвърто, ако дадена операция е недопустима за избраната система за съхранение, това е видимо възможно най-рано — за предпочитане при компилация.

Тези инварианти правят архитектурата едновременно инженерно подредена и академично защитима. Те позволяват всяко следващо разширение да бъде оценявано не само по това дали „работи“, а и по това дали запазва вече формулираните граници.

Следващата глава представя как тези проектни решения се материализират в конкретен код: реализацията на асинхронния слой, конкретните конектори за SQLite, PostgreSQL, MySQL, MongoDB и Redis, и подробната верификация на цялата система.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Настоящата глава описва как проектните решения от Глава III се материализират в конкретен програмен код. Изложението следва три равнища. Първото обхваща реализацията на асинхронния слой и неговите ключови компоненти — `Task<T>`, обединението от работни нишки (`ThreadPool`), `run_on_pool()` и реакторния `IoContext`. Второто описва конкретните реализации на конекторите за основните системи за съхранение (backend): SQLite, PostgreSQL, MySQL, MongoDB и Redis. Третото равнище е верификацията: единични (unit), интеграционни (integration) и реално свързани с живи системи за съхранение тестове, заедно с емпиричната оценка на асинхронния модел.

4.1. Методика и среда за реализация

Реализацията е изградена изцяло върху стандарта C++23 [1], с използване на функционалност за корутини, въведена в C++20 [16, 17, 18]. Кодовата база е организирана по подсистеми. Заглавните файлове са разположени в директории, които съответстват на архитектурните слоеве: `lib/include/ORM/entity/` за обектния слой, `lib/include/ORM/query/` за слоя на заявките, `lib/include/ORM/connector/` за общата фасада и асинхронната обвивка, `lib/include/ORM/async/` за `Task<T>`, `ThreadPool` и `IoContext`, и `lib/include/ORM/db/connectors/` с отделни поддиректории за всяка конкретна система за съхранение. Системата за изграждане е CMake; конструирането и изпълнението на тестовете се извършва чрез `ctest`.

Поддръжката за живи (live) системи за съхранение е условна. Тестовете срещу реален PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra инстанциран сървър се изпълняват само когато съответната среда е активирана чрез променливи на обкръжението и Docker-базирана инфраструктура. Това позволява стандартното изграждане да остане преносимо, докато по-задълбоченото верифициране се извършва селективно. За единичните тестове и базовия интеграционен сценарий този разделителен подход не е необходим — те се компилират и изпълняват винаги.

4.2. Реализация на асинхронния слой

Асинхронният слой е разположен над синхронната фасада `db<DB>` и приема статично конструираното междинно представяне на заявката (`query intermediate`

representation, query IR) като готов вход. Реализацията се фокусира върху три аспекта: жизнения цикъл на корутината (`Task<T>`), универсалния път на изнасяне (`run_on_pool()`) и реакторното обработване на готовността при драйверите с неблокиращи интерфейси (`IoContext`).

4.2.1. `Task<T>`, продължение и завършване

`Task<T>` е централната абстракция в `lib/include/ORM/async/task.h` pp. Имплементацията дефинира `promise_type`, който поддържа: `initial_suspend()`, връщащ `std::suspend_always` (мързеливо стартиране); `final_suspend()`, чрез който се възобновява запазеното продължение (continuation); `return_value(T)` или `return_void()` в зависимост от типа на резултата; и `unhandled_exception()`, който запазва `std::exception_ptr` в обещанието (promise) за по-късно повторно изхвърляне. При изчакване чрез `operator co_await()` текущата корутина се регистрира като продължение на изчакваната задача и се възобновява автоматично, когато тази задача достигне `final_suspend()`.

Помощникът `sync_wait()` осигурява мост към некорутинен код. Той създава нова кратка корутина, регистрирана като продължение, и изпълнява блокираща барутера, докато тази корутина не завърши. Това е важно за единичните тестове и за гранични сценарии, в които приложението още не работи в корутинен стил.

4.2.2. `ThreadPool` и универсален път на изнасяне

`ThreadPool` в `lib/include/ORM/async/thread_pool.hpp` е стандартно реализирано обединение от ограничен брой работни нишки. Той поддържа `std::queue` за задачи, защитен с мутекс, и `std::condition_variable` за уведомяване. Конструкторът приема брой работни нишки и стартира всяка от тях с цикъл, който изважда една задача и я изпълнява до изчерпване на опашката или сигнал за спиране.

Истинският архитектурен мост към корутините се намира в `run_on_pool()`. Тази функция връща обект, удовлетворяващ договора за изчакване (awaitable): `await_ready()` винаги връща `false`; `await_suspend(handle)` публикува извикваемия обект (callable) към обединението и съхранява `handle` плюс мястото за резултата; `await_resume()` извлича резултата (или повдига изключение, ако работата е завършила с грешка). По този начин синхронният код на конектора, който блокира за драйверно действие, се изпълнява в работна нишка, а извикващата корутина се спира и възобновява автоматично, когато

работата приключи. Този универсален път е особено важен за драйвери без неблокиращ интерфейс — честният архитектурен избор е блокиращите извиквания да бъдат изолирани в обединението, а потребителят да получи единен интерфейс, основан на `Task<T>`.

4.2.3. IoContext и интеграция с неблокиращите интерфейси на драйверите

За драйвери с неблокиращ интерфейс библиотеката добавя реакторен слой `IoContext` (`lib/include/ORM/async/io_context.hpp`). Той предоставя `run()`, `run_one()`, `stop()`, `post()`, `schedule()`, `watch_readable(fd)` и `watch_writable(fd)`. Системните примитиви за наблюдение зависят от платформата: на Linux се използва `epoll` [19] или `io_uring` [20], на macOS и BSD — `kqueue` [21], а на Windows — I/O completion ports (IOCP) [22]. Извикванията `watch_readable(fd)` и `watch_writable(fd)` са обекти за изчакване, които не извършват входно-изходно действие; те само спират корутината и я възобновяват, когато съответният файлов описател стане готов.

Реализацията на `AsyncPostgreSQLDB` илюстрира как този слой се комбинира с драйвер, предоставящ неблокиращ интерфейс. Асинхронното свързване използва `PQconnectPoll()`, като корутината последователно изчаква `watch_writable()` или `watch_readable()` според състоянието на крайния автомат на `libpq`. При изпълнение на заявка `PQsendQueryParams()` стартира изпращането, `PQflush()` се извиква в цикъл с изчакване за готовност за запис, а `PQisBusy()` и `PQconsumeInput()` се обединяват с изчакване за готовност за четене.

4.2.4. Обединение от връзки и транзакции при асинхронен достъп

Помощниците `async_connection_pool<DB, N>`, `async_connection_guard<DB>` и `async_transaction_guard<DB>` разширяват синхронния обединителен слой към реализация, интегрирана с корутинния модел. Методът `acquire()` на обединението връща `Task<async_connection_guard<DB>>`. Вътрешно той използва `run_on_pool()` и условна променлива, за да изчака свободен слот, без извикващата корутина да бъде изложена на блокиращ интерфейс. Получената охрана следва идиомата за придобиване на ресурс при инициализация (Resource Acquisition Is Initialization, RAII): при разрушаване тя връща връзката към празно (`idle`) състояние и уведомява чакащите.

`async_begin_transaction()` първо изнася `BEGIN` към обединението и след това връща `async_transaction_guard<DB>`. Методите `commit()` и `rollback()` са `Task<void>`

операции; деструкторът на охраната изпълнява защитно ROLLBACK, ако транзакцията не е финализирана изрично. Този дизайн е едновременно удобен (успешният път е интегриран с корутинния модел) и консервативен (пътят за почистване остава детерминистичен).

4.3. Реализация на конекторите по системи за съхранение

Всеки конкретен конектор е специализация на `connector_trait<DB>`, разположена в собствена поддиректория на `lib/include/ORM/db/connectors/`. Реализациите следват една и съща структура: декларация на маркерите за способности; дефиниция на `wire_type` и `cursor_type`; претоварвания на `execute(...)` за `select_query`, `insert_query`, `update_query` и `delete_query`; и при необходимост — куки за транзакции и за миграция на схемата.

4.3.1. SQLite

SQLiteDB е базовият релационен конектор. Той се реализира директно върху C-интерфейса на SQLite [3], без външни обвивки. Връзката се представя чрез `sqlite3*`, а изпълнението — чрез цикъл от `sqlite3_prepare_v2()`, `sqlite3_bind_*()`, `sqlite3_step()` и `sqlite3_column_*()`. Заместителите се материализират като позиционни въпросителни знаци (?), което съответства напълно на естествения механизъм на драйвера. Конекторът декларира пълен набор от способности за релационен модел: `supports_joins`, `supports_transactions` и `supports_aggregation`.

Изобразяването на междинното представяне към SQL се извършва чрез последователно обхождане на дървото от правила (Rule), полета (field) и заместители (Placeholder). За `select_query` се изграждат фрагменти SELECT, FROM, JOIN, WHERE, GROUP BY, ORDER BY и LIMIT в строго фиксиран ред. Резултатът от изпълнението се материализира в `orm::result<Row, FieldTuple>`, където Row е tuple от извлечените колони, а FieldTuple запазва метаинформацията за проекцията. Така достъпът до полета е възможен както чрез позиционен индекс (`get<I>`), така и чрез указател към член (`get_field<&T::m>`).

SQLite е и референтният сценарий за слоя на миграциите. `SQLiteDB::ddl_for()` обработва `create_table_op`, `add_column_op`, `drop_column_op` и `alter_column_type_op`, превръщайки ги в съответните DDL изрази на езика за описание на данни. Обединението с `migrate<DB>::diff()` дава пълноценен поток за развитие на

схемата, проверен в tests/unit/test_schema_migration.cpp.

4.3.2. PostgreSQL

PostgreSQLDB се реализира върху libpq [4]. Двойната форма PostgreSQLDB и PostgreSQLLiveDB разделя двата сценария: първият е достатъчен за единични и контрактни тестове и не изисква жива (live) база данни, докато вторият работи срещу истински сървър. Заместителите се материализират като \ \$1, \ \$2 и т.н., което съответства на естествения позиционен механизъм на драйвера. Конекторът декларира същия пълен набор от релационни способности като SQLite, но добавя и поддръжка за по-сложни типове и за конкурентно изпълнение чрез supports_concurrent_execute.

Особеност е, че PostgreSQL предоставя неблокиращ интерфейс. Това позволява AsyncPostgreSQLDB да декларира supports_async и да реализира async_execute() чрез IoContext и крайния автомат на libpq. На практика когато потребителят използва async_db<PostgreSQLDB>, статичното разклонение в async_db<DB>::operator<< избира пътя през неблокиращия интерфейс на драйвера вместо изнасяне в обединението от нишки. Това е единственият случай в дипломната работа, в който интеграцията през неблокиращия интерфейс на драйвера е реализирана докрай; за останалите системи за съхранение този път е валиден архитектурно, но реализацията използва обединителния модел.

```
1  template <>
2  struct connector_trait<PostgreSQLDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation    = void;
7      using supports_concurrent_execute = void;
8
9      template <typename T> struct wire_type { using type = T; };
10     using cursor_type = postgresql_cursor;
11
12     template <typename Response, typename... Rest>
13     static auto execute(PostgreSQLDB& db,
14         select_query<Response, Rest...> q, auto&&... params)
15     {
16         auto [sql, n] = render_postgres_sql(q);
17         const char* values[sizeof...(params)];
18         bind_params(values, std::forward<decltype(params)>(params)...);
19
20         PGresult* res = PQexecParams(db.conn(), sql.c_str(),
21             n, nullptr, values, nullptr, nullptr, 0);
```

```

22     return hydrate_result<Response>(res);
23 }
24 };

```

Listing 7: Опростена реализация на `execute` за PostgreSQL

Listing 7 показва, че реализацията не съдържа никаква специфична за приложението логика — тя само изобразява междинното представяне към SQL, свързва параметрите към масива от низове, изисквания от `PQexecParams`, и материализира резултата чрез `hydrate_result<Response>`, която прилага същата проекция, запазена в типа `Response`.

4.3.3. MySQL

MySQLDB следва същата структура като PostgreSQL, но върху MySQL Connector/C [5]. Основната разлика е в модела на свързване на параметри: MySQL използва позиционни въпросителни знаци (?), а не номерирани заместители като PostgreSQL. Това не е трудност за междинното представяне, защото операцията по изобразяване (`render_mysql_sql`) приема един и същ обект, но генерира различен текст на заявката. Реализацията използва крайния автомат на `MYSQL_STMT` за подготовка, свързване и изпълнение, а резултатите се извличат чрез `mysql_stmt_fetch()`.

Маркерите за способности са идентични с тези на PostgreSQL: пълен релационен профил с поддръжка на конкурентно изпълнение. MySQL предоставя неблокиращ интерфейс под формата на двойките `_start/_cont` [8], но реализацията в библиотеката използва пътя с обединението от нишки за момента — архитектурата позволява добавяне на път през неблокиращия интерфейс на драйвера без промяна в публичния интерфейс. Релационната преносимост между PostgreSQL и MySQL се валидира от факта, че един и същ обект на заявката се изпълнява успешно от двата конектора, въпреки диалектните различия.

4.3.4. MongoDB

MongoDB е представителят на документния модел и се реализира върху `libmongoc` [11]. Реализацията изобразява междинното представяне към филтри и проекции в двоично представяне на JSON (Binary JSON, BSON). Полето `field<T::m>` се материализира като име на ключ в документа, а правилата за равенство, неравенство и логическо съчетаване — като операторите `\$eq`, `\$ne`, `\$and` и `\$or`. Заявката `select(...).where(...)` се превръща в извикване на

`mongoc_collection_find_with_opts()`, чийто резултат се материализира в Row чрез последователно извличане на стойностите от документа според проекцията.

`connector_trait<MongoDB>` съзнателно не декларира `supports_joins`, `supports_transactions` или `supports_aggregation` в стиловете на релационния модел. Това не е пропуск, а коректно описание на договора: документната семантика не предлага пряк еквивалент на свързване чрез външен ключ или на SQL агрегация. Отсъствието на тези маркери се проверява изрично в `tests/unit/test_mongodb_connector.cpp`, където опитът за заявка със свързване води до неуспех при компилация чрез `static_assert`.

Релацията със стратегията `store_as::embed` от обектния слой се проявява именно тук: при MongoDB вграждащият вариант се материализира като вложен документ, а препращащият (`store_as::reference`) — като отделен идентификатор, който трябва да бъде извлечен чрез следваща заявка. Това е още един пример за принципа, че логическата форма е обща, но физическата реализация зависи от системата за съхранение.

4.3.5. Redis

RedisDB стъпва върху ключ-стойност модела и се реализира върху hiredis [9]. Обектът на ниво съхранение се картографира по два основни начина според броя на полетата. Когато обектът има едно скалярно поле (или едно сериализирано представяне), той се записва чрез SET key value и се извлича чрез GET key. Когато има няколко полета, се използват хеш-командите HSET key field value и HGETALL key, които позволяват многополева структура в рамките на едно Redis-ключово пространство. Името на логическия контейнер от `table_name_trait<T>` участва в генерирането на префикс за ключа, например `users:42` за обект от тип User с идентификатор 42.

Маркерите за способности са най-ограничителни в цялата библиотека: `connector_trait<RedisDB>` не декларира нито един от четирите релационни маркера (свързвания, транзакции, агрегация, групиране). Това отразява честно естеството на Redis като система за достъп по ключ. Извличането може да се извършва само по първичен ключ; за по-сложни заявки потребителят трябва да поддържа допълнителни ключове за индексация, която не е автоматизирана в библиотеката. Този ограничен договор се валидира от `tests/unit/test_redis_connector.cpp`, където отсъствието на способностите се проверява изрично.

4.3.6. Графов и ширококолонен сценарий (Neo4j и Cassandra)

За пълнота на анализа е важно да се отбележи, че библиотеката съдържа и реализации на Neo4jDB (върху `libneo4j-client` [12]) и CassandraDB (върху драйвера за Cassandra с механизъм на бъдещи стойности [10]). Първият превръща обекти в етикетирани върхове и свойства, а заявките — в MATCH, RETURN и WHERE фрагменти на Cypher. Вторият работи върху Cassandra Query Language (CQL) с особени ограничения, наложени от разделящия (partition) ключ. Подробно описание на тези два конектора не се включва в настоящата глава поради ограничения в обема; те се валидират в `tests/unit/test_neo4j_connector.cpp` и `tests/unit/test_cassandra_connector.cpp` съответно. Същевременно техните маркери за способности са включени в обобщителната Таблица 6.

4.3.7. Обобщителна матрица на способностите

Реализациите по системи за съхранение могат да бъдат сведени до компактна матрица на способностите. Тя описва архитектурната позиция на библиотеката спрямо съответната система за съхранение в текущата ревизия на хранилището, а не универсална истина за самите бази данни.

Таблица 6. Способности на основните системи за съхранение според специализациите и тестовете

Backend	Свър.	Транз.	Агр.	Upsert	Масово	Коментар
SQLite	да	да	да	ограничен	ограничен	пълнен релационен референтен път
PostgreSQL	да	да	да	специфичен	възможен	силен релационен договор с жива проверка
MySQL	да	да	да	специфичен	възможен	оперативен път с диалектни различия
MongoDB	не	не	не	аналог	не е централно	документен модел без релационни маркери
Redis	не	не	не	аналог	ограничен	ключ-стойност с минималистичен договор
Neo4j	аналог	да	не	не е централно	не е централно	графова система с акцент върху транзакциите
Cassandra	не	да	не	не е централно	възможен	ширококолонен с ограничен набор от способности

Таблица 6 показва, че контролираната нееднородност е централна за дизайна. Релационните системи за съхранение концентрират по-богат набор от способности, а документните, графовите и ключ-стойност вариантите се оценяват през други критерии. Това е инженерно честното положение — библиотеката не обещава „универсална база данни през един интерфейс“, а *унифициран логически слой с явни граници спрямо системата за съхранение*.

4.4. Верификация и тестване

Верификационната стратегия следва архитектурата на библиотеката. Слойт, проверяван по време на компилация, се валидира чрез концепции, ограничения чрез маркери за способности и типизирано изграждане на междинното представяне. Слойт, проверяван по време на изпълнение, се покрива от единични тестове за изобразяване (rendering), за обвързване на параметри, за нишкова безопасност, за миграции, за

корутинната инфраструктура и за асинхронния достъп. Най-външният слой се валидира от интеграционни тестове срещу реални системи за съхранение, така че се избягва смесване на коректността на имитиращи (mock) обекти с реалното поведение на драйверите. В текущата ревизия пълният автоматизиран набор включва 271 теста при изпълнение чрез `cctest`.

4.4.1. Стратегия на верификацията

Стратегията се основава на принципа, че всяко архитектурно твърдение трябва да има поне един пряк тестов корелат. Това включва както позитивни проверки (декларираните способности водят до успешно изпълнение на съответните операции), така и негативни проверки (отсъствието на маркер за свързване води до отказ при компилация). Негативната верификация е особено важна за конекторите MongoDB и Redis — именно тя гарантира, че договорът на конектора отразява честно неговите ограничения, а не само неговите възможности.

Таблица 7. Основни източници на верификационни доказателства

Източник	Какво валидира	Представителни артефакти
Ограничения при компилация и междинно представяне	типова коректност, маркери за способности и договори на конекторите	tests/unit/test_query.cpp, tests/unit/test_postgresql_connector.cpp, tests/unit/test_mongodb_connector.cpp
Асинхронна инфраструктура	Task<T>, ThreadPool, IoContext, async_db<DB>, обединение и транзакции	tests/unit/test_task.cpp, tests/unit/test_thread_pool.cpp, tests/unit/test_io_context.cpp, tests/unit/test_async_connector.cpp
Миграции и нишкова безопасност	разлика в схемите, отмяна, охрана на връзките и конкурентна безопасност	tests/unit/test_schema_migration.cpp, tests/unit/test_thread_safety.cpp
Живи интеграции на системите за съхранение	поведение върху реални SQLite, PostgreSQL, MySQL, MongoDB и Redis среди	tests/integration/test_sqlite.cpp, tests/integration/test_postgresql_live.cpp, tests/integration/test_mongodb_live.cpp, tests/integration/test_redis_live.cpp
Емпирична оценка	пропускливост (throughput) при симулирано изчакване и изолирани измервания на режимното време (overhead)	benchmarks/bench_async.cpp

4.4.2. Каталог на единичните и интеграционните тестове

Единичните тестове покриват всички подсистеми на библиотеката: типовете и обектите (test_property.cpp, test_relationship.cpp); средното представяне на заявката (test_query.cpp); миграциите (test_schema_migration.cpp); нишковата безопасност (test_thread_safety.cpp); и асинхронната инфраструктура (test_task.cpp, test_thread_pool.cpp, test_io_context.cpp, test_async_connector.cpp). Допълнително test_cpp26_reflection.cpp експериментира с ранен път за интеграция с бъдещата функционалност за отражение (reflection) на C++26.

Контрактните тестове по системи за съхранение (Таблица 8) валидират както позитивни, така и негативни способности. Те са разположени в `tests/unit/` и не изискват жива база данни — използват имитиращи обекти и проверки чрез `static_assert`, които установяват, че опитът да се използва неподдържана операция не преминава компилацията.

Таблица 8. Контрактни тестове по системи за съхранение

Файл	Фокус върху системата за съхранение	Тип доказателство
<code>test_postgresql_connector.cpp</code>	PostgreSQLDB	съвместимост с <code>is_connector</code> , проверки на способностите и изобразяване на параметри
<code>test_mysql_connector.cpp</code>	MySQLDB	договор за свързвания и транзакции, диалектни особености и очаквания при обвързване
<code>test_mongodb_connector.cpp</code>	MongoDB	негативна валидация на способностите и изобразяване на филтри в BSON
<code>test_redis_connector.cpp</code>	RedisDB	отсъствие на свързвания, транзакции и агрегация; материализация на команди
<code>test_cassandra_connector.cpp</code>	CassandraDB	ограничения на способностите и съответствия с CQL
<code>test_neo4j_connector.cpp</code>	Neo4jDB	транзакционна способност и графова материализация на заявки

4.4.3. Интеграционни и реално свързани с живи системи за съхранение тестове

Интеграционните тестове са разделени на два слоя. Първият е `tests/integration/test_mockdb.cpp` — винаги наличен интеграционен базов сценарий, който проверява съставянето на заявки, обвързването на параметрите, подготвените заявки и CRUD сценариите без външна система за съхранение. Той е особено полезен, защото гарантира, че логическият слой е стабилен независимо от наличието на жива база данни. Вторият слой включва тестовете срещу реални системи за съхранение, които се

активират условно: `test_sqlite.cpp` (локален интеграционен тест, винаги наличен), `test_postgresql_live.cpp`, `test_mysql_live.cpp`, `test_mongodb_live.cpp`, `test_redis_live.cpp`, `test_neo4j_live.cpp` и `test_cassandra_live.cpp`.

Условното активиране се извършва чрез променливи на обкръжението: ако съответната променлива (например `ORM_TEST_POSTGRESQL=1`) е зададена, тестът се изпълнява срещу инстанция, предоставена от Docker-базирана инфраструктура. Това позволява разработчикът да изпълнява пълния набор от тестове в среда без външни зависимости, а екипите за качество да активират по-задълбочените проверки в специално подготвена среда. Разделението избягва смесване на коректността на имитиращи обекти с действителното поведение на драйверите.

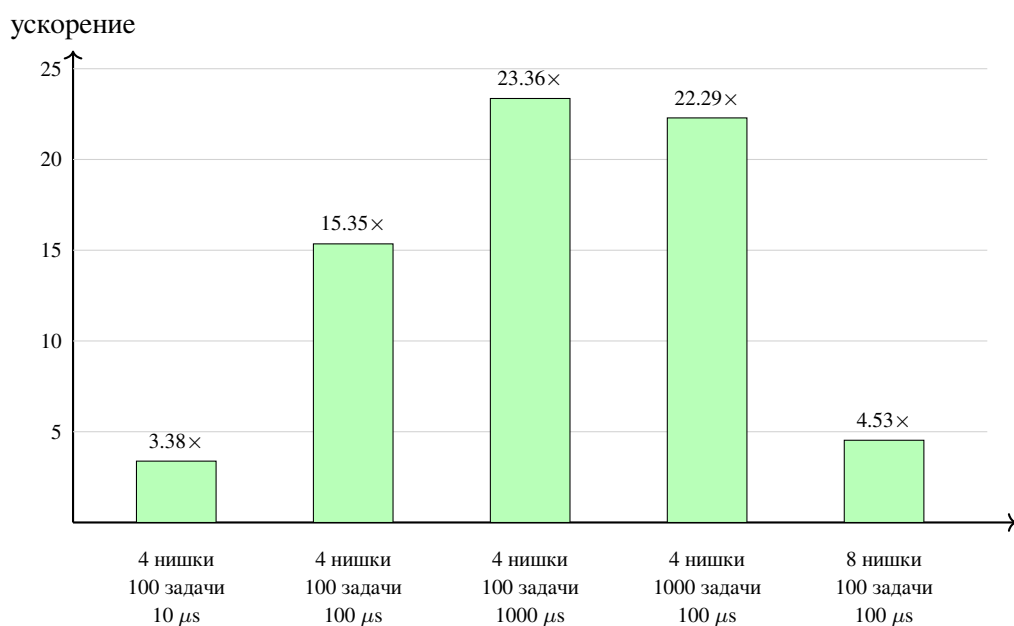
4.4.4. Емпирични резултати

Емпиричната оценка използва библиотеката Google Benchmark [23] и е реализирана в `benchmarks/bench_async.cpp`. Тя не симулира пълния жизнен цикъл на конкретна база данни, а изолира архитектурния въпрос дали корутинно базираното изпълнение освобождава работните нишки по-ефективно от синхронното блокиране, когато между две кратки изчислителни фази съществува интервал на изчакване. Наборът е разделен на две групи. Първата измерва пропускливост (`throughput`) на цели партии от задачи: синхронният базов сценарий блокира работна нишка, докато асинхронният използва `SimulatedAsyncIO` обект за изчакване, който спира корутината, планира отложено възобновяване чрез таймерна опашка и връща продължението обратно към обединението от нишки. Втората група измерва изолирани режимни времена: създаване и завършване на `Task<int>`, обхождане в две посоки на `ThreadPool::post()` и вложени вериги от `co_await`.

Таблица 9. Представителни резултати в режим release

Нишки	Задачи	Изчакване	Синхронна	Асинхронна	Ускорение
			пропускливост	пропускливост	
4	100	10 μs	194,6 k оп/с	658,5 k оп/с	3.38×
4	100	100 μs	29,8 k оп/с	458,2 k оп/с	15.35×
4	100	1000 μs	3,22 k оп/с	75,28 k оп/с	23.36×
4	1000	100 μs	30,08 k оп/с	670,5 k оп/с	22.29×
8	100	100 μs	57,13 k оп/с	258,5 k оп/с	4.53×

Най-важният извод от Таблица 9 е, че предимството на корутинния модел нараства не когато локалната изчислителна работа се увеличи, а когато фазата на изчакване започне да доминира над нея. При 4 работни нишки и 100 задачи разликата между 10 и 1000 микросекунди изчакване увеличава относителното предимство на асинхронния вариант от 3.38× до 23.36×. Това е точно очакваното поведение: при синхронния базов сценарий нишките се изчерпват от блокиране, докато при корутинния модел те се освобождават за друга работа. Вторият извод е, че ефектът се запазва и при по-големи партии — при 4 нишки, 1000 задачи и 100 μs изчакване асинхронният вариант достига 670,5 k операции в секунда срещу 30,08 k при синхронния.



Фигура 5. Относително ускорение на асинхронната пропускливост спрямо синхронната

При нулева изкуствена фаза на изчакване (Таблица 10) ситуацията се променя. Пропускливостта на асинхронния вариант е сравнима или дори малко по-ниска от синхронната. Това е важен коректив: корутинният слой не „създава“ пропускливост сам по себе си — той добавя малко, но измеримо допълнително време, което може да бъде компенсирано само частично, когато няма реално изчакване.

Таблица 10. Представителни резултати при нулева фаза на изчакване

Нишки	Задачи	Синхронна пропускливост	Асинхронна пропускливост	Отношение асинх./синхр.
4	100	841,0 k оп/с	989,4 k оп/с	1.18×
4	1000	1,232 М оп/с	1,184 М оп/с	0.96×
8	100	810,9 k оп/с	514,4 k оп/с	0.63×

Изолираните измервания показват, че режимното време на корутинната инфраструктура не доминира в сценариите, ограничени от вход-изход. В режим `release` `BM_TaskCreateAndWait` измерва приблизително 71,6 ns за създаване и завършване на тривиална задача, `BM_ThreadPoolPost` остава около 5,0 μ s независимо от размера на обединението от нишки, а `BM_NestedCoAwait` достига около 324 ns при дълбочина 10 и около 2,375 μ s при дълбочина 100. Тези числа са с порядъци по-малки от 100–1000 μ s фази на изчакване, при които асинхронната пропускливост печели най-много. С други думи, наблюдаваното ускорение не е артефакт на измервателния инструмент, а следствие от това, че работните нишки престават да бъдат заключени в блокиращо изчакване.

4.4.5. Покритие и ограничения на верификацията

Емпиричната оценка съзнателно използва симулирана фаза на изчакване, а не пълно изпълнение от край до край срещу мрежов сървър за база данни. Това е ограничение, ако целта е да се прогнозира абсолютна пропускливост за реална инсталация на PostgreSQL или Cassandra, но е методологично предимство, ако целта е да се изолира моделът на изпълнение от влиянието на планировчика на заявките, мрежовите колебания, кеширането в драйвера и спецификите на сторидж двигателя. Следователно представените резултати не доказват, че „асинхронното е винаги по-бързо“; те доказват по-тясната, но по-научно защитима теза, че когато архитектурата работи в режим, ограничен от вход-изход, и фазата на изчакване доминира над локалната изчислителна работа, корутинно базираният модел

позволява съществено по-добро използване на работните ресурси от синхронния базов сценарий.

Тестовата верификация и емпиричната оценка покриват двата основни въпроса на дипломната работа. От една страна библиотеката показва формална и тестово подкрепена коректност. От друга страна асинхронният изпълнителен модел показва измерима практическа полза точно в сценария, за който е проектиран. Така емпиричната оценка не стои встрани от архитектурата, а потвърждава нейния централен инженерен и научен смисъл.

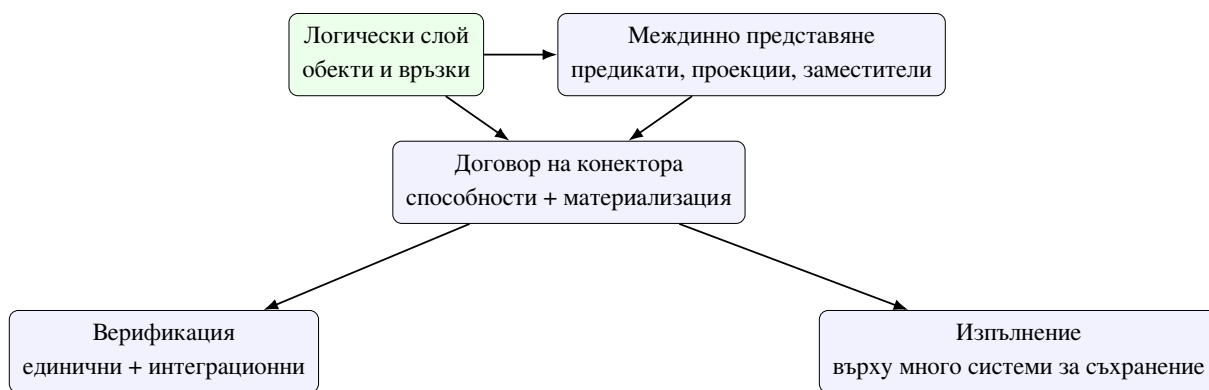
V. ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа представи проектирането, реализацията и анализа на библиотека за обектно-релационно картографиране по време на компилация (compile-time Object-Relational Mapping) за C++, която обхваща както релационни, така и нерелационни системи за съхранение. Централната ѝ идея е, че логическият модел на данните, структурата на заявките и съществена част от ограниченията върху операциите могат да бъдат описани и проверени чрез типовата система на езика, а не чрез механизми, действащи по време на изпълнение.

5.1. Обобщение на решената задача

Поставената задача не беше просто да се изгради удобен съставител на заявки. Целта беше да се проектира библиотека, в която логическият модел на данните се описва чрез C++ типове; заявките се представят като типизирано междинно представяне, а не като низове; конкретните системи за съхранение се интегрират чрез формален договор; различията между моделите за съхранение се управляват чрез механизъм на способностите и компилационни ограничения; и коректността се подкрепя едновременно от статични проверки и от тестова инфраструктура.

В рамките на работата всяка от тези подцели е адресирана с конкретни архитектурни и реализационни решения. Обектният слой въведе типизирани свойства и връзки, политика на съхранение чрез `store_as` и обобщено именуване на логическите контейнери. Слойът на заявките се организира около типизирано междинно представяне, а слойът на конекторите се формализира чрез `connector_trait<DB>` и маркери за способности. Асинхронната подсистема, изградена върху корутини, разшири модела с честно разграничение между универсалния път на изнасяне в обединение от нишки и интеграцията през неблокиращите интерфейси на драйверите посредством реакторен слой. Верификационната стратегия е разпределена между ограниченията при компилация, единичните и интеграционните тестове и емпиричната оценка на асинхронния модел.



Фигура 6. Синтез на основната конструкция на разработената библиотека

Фигура 6 обобщава цялостната логика на работата. Тя показва, че библиотеката не е съвкупност от отделни техники, а последователна система, в която логическият слой, междинното представяне, договърът на конектора, верификацията и изпълнението върху много системи за съхранение се поддържат взаимно.

5.2. Постигнати научни и приложни приноси

Първият принос на работата е демонстрацията, че практически полезна библиотека за обектно-релационно картографиране може да бъде реализирана в стандартен C++ без външна кодогенерация, без виртуален интерфейс на конектора и без принуда потребителят да напуска идиоматичния модел на езика.

Вторият принос е изясняването на връзката между описанието на обектите в C++ и физическата организация на данните. Показано е, че моделът в C++ може да се третира като първичен носител на логическата схема, докато конкретната система за съхранение определя начина на материализация. Тази позиция има стойност не само за библиотеките за обектно-релационно картографиране, но и за по-широк клас системи, които комбинират силна статична типизация с разнородни източници на данни.

Третият принос е формализирането на договора на конектора, който обединява разширяемостта и строгата компилационна дисциплина. Той прави възможно надграждане с нови системи за съхранение, без да се разрушават вече съществуващите гаранции на интерфейса. Точно тук работата се различава от подходите, които предлагат общ интерфейс, но скриват критичните разлики между системите за съхранение.

Четвъртият принос е изграждането на асинхронен изпълнителен слой върху корутините на C++20. Той показва как статично формализираният достъп до данни

може да бъде изпълняван през два честно разграничени асинхронни пътя — универсален (изнасяне в обединение от нишки) и през неблокиращия интерфейс на драйвера (реакторен слой), — без библиотеката да губи типова дисциплина, безопасност на жизнения цикъл или прозрачност на конекторите. Емпиричната оценка показва, че този слой носи измерима полза в режим, ограничен от вход-изход (например ускорение от $23.36\times$ при 4 нишки, 100 задачи и $1000\ \mu s$ изчакване).

Таблица 11. Връзка между целите на дипломната работа и постигнатите резултати

Цел	Реализиран механизъм	Доказателствена опора
Типизиран модел на данните	property, relationship, table_name_trait	заглавните файлове на обектния слой; единичните тестове за свойства и връзки
Типизиран слой на заявките	съставители и междинно представяне	файловете на слоя на заявките, tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp
Разширяем слой за много системи за съхранение	договор на конектора и маркери за способности	заглавните файлове на конектора и тестовите по системи за съхранение
Подход, работещ със схемата	migrate<DB> и куки за DDL	tests/unit/test_schema_migration.cpp
Безопасна оперативна употреба	помощници за нишкова безопасност и транзакции	tests/unit/test_thread_safety.cpp
Асинхронно изпълнение	Task<T>, ThreadPool, IoContext, async_db<DB>	асинхронните единични тестове и benchmarks/bench_async.cpp

Таблица 11 показва, че работата не остава на равнището на декларативен проектен документ. За всяка основна цел има конкретен реализиран механизъм и поне един пряк технически корелат в хранилището.

5.3. Ограничения на разработката

Работата ясно показва, че абстракцията за много системи за съхранение има граници. Не всеки модел за съхранение предлага едни и същи операции, а следователно общото междинно представяне на заявката не може да бъде интерпретирано еднакво навсякъде. Точно затова механизмът на способностите е толкова важен — той не отслабва библиотеката, а я прави коректна спрямо разликите между системите за съхранение.

Допълнително, макар архитектурната основа за миграции, нишкова безопасност и интеграция през неблокиращите интерфейси на драйверите вече да съществува, тези подсистеми остават естествено поле за по-нататъшно развитие. Интеграцията през неблокиращия интерфейс на драйвера е реализирана докрай само за PostgreSQL; за останалите системи за съхранение универсалното изнасяне в обединение от нишки е честен и адекватен заместител, но при по-зрели драйверни интерфейси то би могло да бъде заменено с път през неблокиращия интерфейс на драйвера. Емпиричната оценка използва симулирана фаза на изчакване, а не пълно изпълнение от край до край срещу мрежов сървър — това позволява чисто сравнение между моделите на изпълнение, но не прогнозира абсолютна пропускливост в производствена среда.

От академична гледна точка е важно да се подчертае, че работата не доказва универсална превъзходност на статичния подход за всички приложения. Тя доказва нещо по-точно: за клас системи с нужда от ясна статична структура, висока типова безопасност и контролирана разширяемост към много системи за съхранение този подход е жизнеспособен и инженерно защитим.

5.4. Бъдещо развитие

Бъдещото развитие следва пряко от вече реализираните архитектурни решения и може да бъде групирено в няколко стратегически направления (Таблица 12).

Таблица 12. Приоритетни направления за бъдещо развитие

Направление	Засегнати подсистеми	Очакван ефект	Предизвикателство
По-зряло покритие на живи системи за съхранение	конектори, интеграционни тестове, пътища при грешка	по-висока практическа надеждност	различна оперативна семантика между системите за съхранение
Автоматизация на работата със схемата	migrate<DB>, куки за DDL, метаданни	по-близка до производствено приложение интеграция	съчетаване на компилационна и изпълнителна истина
Неблокираща интеграция чрез интерфейсите на драйверите	реакторен слой и крайни автомати на драйверите	по-нисък разход в реални мрежови сценарии	различни модели на готовност при различните драйвери
Моделиране, базирано на отражение (reflection)	обектен слой, извличане на метаданни, ергономия на интерфейса	по-малко шаблонен код и подход „модел напред“	баланс между автоматизация и прозрачност
По-фина таксономия на способностите	connector_trait<DB> и статични проверки	по-точен договор за нови системи за съхранение	избягване на декоративни маркери
По-богата емпирична оценка	асинхронен слой и подготвени заявки	по-точен модел на разходите	необходимост от стабилни производствени среди

Първото направление е по-голяма оперативна зрелост на вече наличните системи за съхранение: по-широко покриване на типовете, диагностичните пътища при грешка, политиките за жизнен цикъл на връзките и поведението при по-сложни заявки. Второто е по-пълна автоматизация на схемата — извличане на живи метаданни от драйверите, генериране на безопасни DDL операции и анализ на съвместимостта. Третото е

разширяване на интеграцията през неблокиращите интерфейси на драйверите и за други системи за съхранение (MySQL чрез `_start/_cont` двойките, Redis чрез `hiredis` с обратни извиквания), което би доближило библиотеката до по-нисък разход в реални мрежови сценарии. Четвъртото е интегриране на бъдещата функционалност за отражение (reflection) на C++26, която би позволила още по-кратко описание на обектите. Петото е по-фина таксономия на способностите — например разграничаване между безопасно паралелно четене и пълно конкурентно изпълнение, между специфични форми на вграждане в документ или между различни нива на поддръжка на стрийминг резултати.

Накрая, една бъдеща версия на работата следва да включва по-систематичен набор от измервания, в който се сравняват не само различните системи за съхранение, но и различни режими на изпълнение в рамките на самата библиотека — пряко изпълнение, повторна употреба на подготвени заявки, сценарии за масово вмъкване и различни форми на материализация на резултата. Такива измервания биха показали реалната цена на различните архитектурни избори и биха обогатили както инженерната, така и научната оценка на разработката.

5.5. Значение и заключителни бележки

Практическата стойност на разработката се състои в това, че библиотеката предоставя стабилна основа за по-нататъшно инженерно развитие. Обектното описание, междинното представяне на заявката, договорът на конектора и верификационната стратегия са достатъчно ясно формулирани, за да позволят дисциплинирано добавяне на нови системи за съхранение и оперативни възможности.

Изследователската стойност е свързана с по-общия въпрос за ролята на типовата система в моделирането на достъпа до данни. Работата показва, че типовата система на C++ може да служи не само за описание на данни и алгоритми, но и за формализиране на архитектурните граници, допустимостта на операциите и прехода между логически и физически слоеве. Това поставя библиотеката в по-широк контекст, който надхвърля темата за обектно-релационното картографиране в тесния смисъл. Комбинацията от статичен обектен модел, типизирано междинно представяне, договор на конектора, основан на признаци (traits), асинхронен слой над корутини и систематична верификация показва устойчив път към библиотеки, които са едновременно изразителни, безопасни и разширяеми.

По този начин дипломната работа постига както инженерна, така и изследователска цел: изгражда работеща библиотека и едновременно формулира принципи, които могат да бъдат използвани при бъдещи разработки на слоеве за достъп до данни по време на компилация. Най-същественият извод е, че този подход не е теоретично упражнение, а реална инженерна алтернатива за системите, които изискват строгост, яснота и разширяемост при работа с множество модели за съхранение.

ЛІТЕРАТУРА

- [1] ISO/IEC JTC1/SC22/WG21, “ISO/IEC 14882:2024 – programming language C++,” international standard, International Organization for Standardization, 2024.
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] D. R. Hipp, “SQLite C/C++ interface – an introduction.” <https://www.sqlite.org/cintro.html>, 2000. Online documentation.
- [4] The PostgreSQL Global Development Group, “libpq — C library for PostgreSQL.” <https://www.postgresql.org/docs/current/libpq.html>, 1996. Online documentation.
- [5] Oracle Corporation, “MySQL connector/C – C client library for MySQL.” <https://dev.mysql.com/doc/c-api/en/>, 2000. Online documentation.
- [6] D. R. Hipp, “SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine.” <https://www.sqlite.org/>, 2000. Online documentation.
- [7] The PostgreSQL Global Development Group, “PostgreSQL documentation: Asynchronous command processing in libpq.” <https://www.postgresql.org/docs/current/libpq-async.html>, 2026. Online documentation, accessed 2026-04-06.
- [8] Oracle Corporation, “MySQL C api developer guide: C api asynchronous interface.” <https://dev.mysql.com/doc/c-api/8.4/en/c-api-asynchronous-interface.html>, 2026. Online documentation, accessed 2026-04-06.
- [9] Redis Ltd. and hiredis contributors, “hiredis — minimalistic C client library for Redis.” <https://github.com/redis/hiredis>, 2026. GitHub repository and documentation, accessed 2026-04-06.
- [10] DataStax, “DataStax C/C++ driver: Futures.” <https://datastax.github.io/cpp-driver/2.6/topics/basics/futures/>, 2026. Online documentation, accessed 2026-04-06.
- [11] MongoDB, Inc., “libmongoc: mongoc_client_pool_t.” https://mongoc.org/libmongoc/current/mongoc_client_pool_t.html, 2026. Online documentation, accessed 2026-04-06.
- [12] C. Leishman, “libneo4j-client — C client library for Neo4j.” <https://neo4j-client.net/>, 2026. Online documentation, accessed 2026-04-06.

- [13] M. Hryniuk and M. Bao, “SOI: The c++ database access library.” <https://soci.sourceforge.net/>, 2004. Online documentation.
- [14] R. Witt, “sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++.” <https://github.com/rbock/sqlpp11>, 2014. GitHub repository.
- [15] Code Synthesis Tools CC, “ODB: C++ object persistence framework.” <https://www.codesynthesis.com/products/odb/>, 2010. Online documentation.
- [16] ISO/IEC JTC1/SC22/WG21, “ISO/IEC 14882:2020 – programming language C++,” international standard, International Organization for Standardization, 2020.
- [17] ISO/IEC JTC1/SC22/WG21, “P0912R5: Merge coroutines TS into C++20 working draft.” <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0912r5.html>, 2019. ISO/IEC JTC1/SC22/WG21 proposal.
- [18] ISO/IEC JTC1/SC22/WG21, “P1063R0: Core coroutines.” <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1063r0.pdf>, 2018. ISO/IEC JTC1/SC22/WG21 proposal.
- [19] The Linux Kernel Community, “epoll(7) — i/o event notification facility.” <https://man7.org/linux/man-pages/man7/epoll.7.html>, 2024. Linux manual page, accessed 2026-04-06.
- [20] J. Axboe, “Efficient IO with io_uring.” https://kernel.dk/io_uring.pdf, 2019. Technical paper.
- [21] Apple Inc., “kqueue(2) — kernel event notification mechanism.” https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man2/kqueue.2.html, 2009. Manual page.
- [22] Microsoft Corporation, “I/O completion ports.” <https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports>, 2024. Microsoft Learn documentation, accessed 2026-04-06.
- [23] Google, “Google Benchmark user guide.” https://google.github.io/benchmark/user_guide.html, 2026. Online documentation, accessed 2026-04-06.